CIRCLEGUARD · INGENIERÍA DE SOFTWARE V

Análisis de estructura del microservicio Gateway

Diagnóstico técnico de **circleguard-gateway-service**: estructura, dependencias, integraciones, pruebas y hallazgos.

Documento	Análisis de estructura y diagnóstico técnico
Microservicio	circleguard-gateway-service
Puerto	8087
Repositorio	circle-guard-finalproject (monorepo Gradle)
Stack	Java 21 · Spring Boot 3.2.4 · Redis · JJWT
Fecha	6 de noviembre de 2026

Alcance del documento

Revisión estática del código fuente, configuración, pruebas e integraciones del servicio. No se ejecutaron los servicios ni se modificó el repositorio. Las referencias a archivos y líneas corresponden al estado actual del proyecto.

1. Resumen ejecutivo

El microservicio **Gateway** es el componente más pequeño de los ocho servicios de CircleGuard. Su función no es enrutar tráfico como un *API Gateway* de borde, sino **validar el ingreso al campus**: recibe un código QR firmado (JWT), comprueba su firma y vigencia, consulta en Redis el estado de salud del identificador anónimo y responde si el acceso se permite (GREEN) o se deniega (RED).

La implementación actual es un esqueleto funcional y muy compacto: tres clases Java, una sola ruta HTTP y dos pruebas unitarias de servicio más una de controlador. Depende únicamente de **Redis** y de la librería **JJWT**; no usa base de datos relacional, Neo4j ni Kafka, y no llama por HTTP a otros servicios.

Sin embargo, el análisis revela **problemas críticos de correctitud y seguridad** que impiden considerarlo listo para producción. El más grave es que el gateway deniega los estados `CONTAGIED` y `POTENTIAL`, mientras que `promotion-service` escribe `SUSPECT`, `PROBABLE` y `CONFIRMED`. En consecuencia, un caso confirmado o un contacto de riesgo recibiría **GREEN**. Además, un estado ausente en Redis también permite el paso, el endpoint no exige autenticación y el QR configurado en auth expira en 300 milisegundos, no en 300 segundos.

> **Conclusión anticipada**
>
> El servicio cumple el contrato básico de validación, pero su lógica de decisión está desalineada con el vocabulario de estados del resto del sistema. Corregir esa matriz de decisión, el TTL del QR y la autenticación del endpoint es obligatorio antes de desplegarlo en cualquier ambiente compartido.

2. Ubicación y composición

El servicio se encuentra en `services/circleguard-gateway-service` y está registrado en `settings.gradle.kts` como `:services:circleguard-gateway-service`. Su código principal ocupa unas 80 líneas y se organiza en tres capas convencionales de Spring Boot.

```
services/circleguard-gateway-service/
|-- build.gradle.kts
|-- bin/main/application.yml          <-- artefacto de compilacion versionado
`-- src/
    |-- main/
    |   |-- java/com/circleguard/gateway/
    |   |   |-- GatewayServiceApplication.java
    |   |   |-- controller/GateController.java
    |   |   `-- service/QrValidationService.java
    |   `-- resources/application.yml
    `-- test/java/com/circleguard/gateway/
        |-- controller/GateControllerTest.java
        `-- service/QrValidationServiceTest.java
```

Archivo	Líneas	Responsabilidad
GatewayServiceApplication.java	11	Punto de entrada de Spring Boot; habilita el escaneo de componentes.
GateController.java	20	Expone la ruta HTTP de validación y delega en el servicio.
QrValidationService.java	49	Valida el JWT, consulta Redis y construye el resultado.
application.yml	18	Puerto, conexión Redis y secretos de configuración.
build.gradle.kts	18	Dependencias: Web, Redis, JJWT y pruebas.

Tabla 1. Composición del microservicio.

No existen paquetes `config`, `dto`, `security`, `repository`, `client` ni `exception`. Toda la lógica de negocio vive en una única clase de servicio y el contrato de salida se define con un *record* interno.

3. Arquitectura y flujo de validación

El flujo es **sin estado** y síncrono. El personal de portería escanea el QR del usuario con el cliente Expo; el cliente envía el token al gateway; el gateway verifica la firma HS256 con el secreto compartido y consulta el estado de salud en Redis, que previamente fue materializado por `promotion-service`.

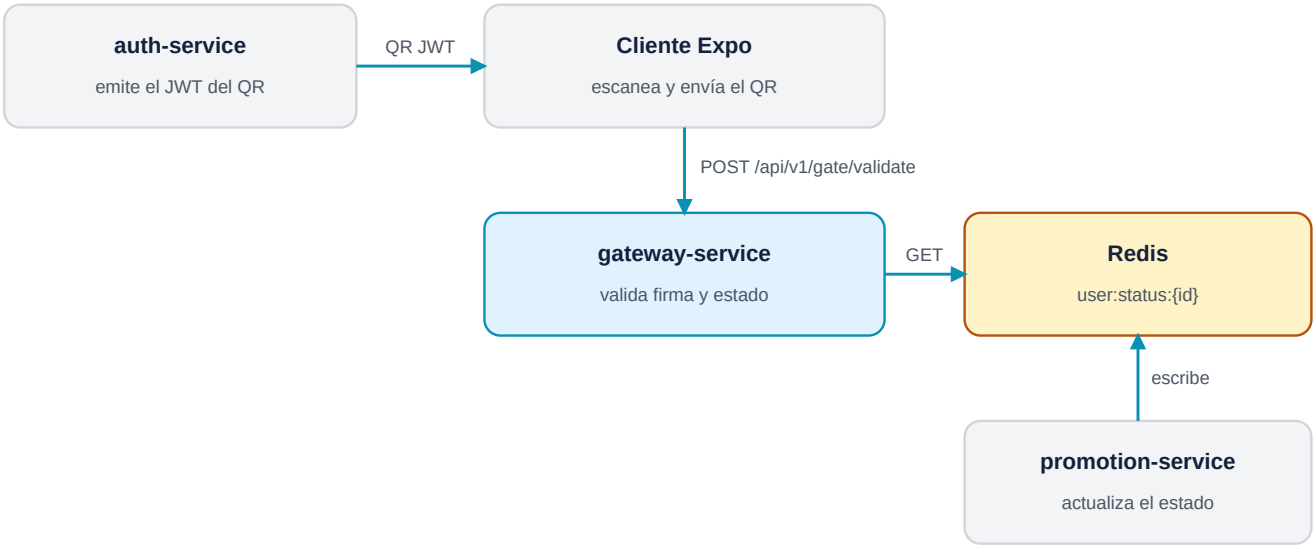


Figura 1. Flujo de validación de ingreso y fuentes del estado de salud.

Conviene precisar la nomenclatura: pese a llamarse *Gateway*, el servicio **no implementa el patrón API Gateway**. No hay Spring Cloud Gateway, enrutamiento, balanceo, terminación TLS, filtros transversales ni agregación de APIs. Es un **servicio de dominio** para control de acceso físico. Esta distinción es relevante para el entregable E2.4, donde se pide identificar patrones y verificar su presencia en el código.

Patrones observables en el código

- **Arquitectura en capas:** controlador → servicio, con responsabilidades separadas.
- **Validación stateless con JWT:** el token contiene el identificador anónimo y se verifica con HMAC-SHA256; no se guarda sesión.
- **Caché compartida / vista materializada:** el gateway consume en Redis el estado que escribe promotion-service, evitando una consulta síncrona a Neo4j en la portería.
- **Configuración externa:** puerto, Redis y secretos se leen desde `application.yml` y `@Value`.
- **DTO inmutable:** `ValidationResult` es un *record* con tres campos.
- **Fail-closed grueso:** cualquier excepción termina en denegación, aunque sin distinguir la causa.

4. Endpoint y contrato

Existe una única operación HTTP. No hay versionado por cabecera ni documentación OpenAPI generada.

```
POST /api/v1/gate/validate
Content-Type: application/json
Authorization: Bearer <jwt-del-empleado>   (enviado por el cliente, ignorado por el gateway)

{ "token": "<jwt-qr>" }
```

Caso	HTTP	Cuerpo de respuesta
Permitido	200	{ "valid": true, "status": "GREEN", "message": "Welcome to Campus" }
Riesgo sanitario	200	{ "valid": false, "status": "RED", "message": "Access Denied: Health Risk Detected" }
Token inválido o expirado	200	{ "valid": false, "status": "RED", "message": "Invalid or Expired Token" }

Tabla 2. Contrato actual de respuesta.

El servicio siempre responde **HTTP 200**, incluso cuando niega el acceso o cuando el token es inválido. La decisión se comunica en el campo `valid`. Esto simplifica al cliente, pero impide usar códigos semánticos (401, 403, 503) y dificulta el monitoreo y la observabilidad.

5. Configuración

Propiedad	Valor	Observación
server.port	8087	Puerto HTTP del servicio; coincide con el cliente Expo.
spring.data.redis.host	localhost	Host de Redis; sin TLS ni autenticación.
spring.data.redis.port	6379	Puerto estándar de Redis.
qr.secret	texto plano	Secreto HMAC compartido con auth-service; versionado en el repositorio.
qr.expiration	300	Definido pero **no utilizado** en el gateway.
jwt.secret / jwt.expiration	texto plano	Definidos pero **no utilizados** en este servicio.

Tabla 3. Configuración efectiva del servicio.

La configuración no está externalizada por ambiente: no hay variables de entorno, perfiles ni integración con un gestor de secretos. El archivo `bin/main/application.yml` es un artefacto de compilación que quedó versionado y duplica la configuración de `src/main/resources`.

6. Dependencias

Dependencia	Propósito
spring-boot-starter-web	Servidor embebido y anotaciones REST (@RestController, @PostMapping).
spring-boot-starter-data-redis	StringRedisTemplate para leer el estado de salud.
jjwt-api / jjwt-impl / jjwt-jackson	Construcción y verificación de JWT con HMAC-SHA256.
spring-boot-starter-test	JUnit 5, Mockito, MockMvc y aserciones.
Lombok (heredado del build raíz)	Genera el constructor de QrValidationService mediante @RequiredArgsConstructor.

Tabla 4. Dependencias declaradas en build.gradle.kts.

El módulo aplica los plugins de Kotlin (`kotlin("jvm")` y `kotlin("plugin.spring")`) aunque todo su código es Java. Funciona porque el build raíz aplica también el plugin `java` a todos los subproyectos, pero añade complejidad innecesaria. Tampoco declara **Spring Boot Actuator**, **Security**, **Validation**, **Resilience4j** ni **OpenAPI**.

7. Integraciones con otros componentes

Componente	Rol	Detalle
auth-service	Emite el QR	GET /api/v1/auth/qr/generate genera un JWT con el mismo qr.secret y el anonymousId como subject.
promotion-service	Escribe el estado	Publica en Redis las claves user:status:{anonymousId} tras cada promoción o resolución.
Cliente Expo	Consume la API	useGateValidation.ts envía el token escaneado y un Bearer que el gateway no valida.
Redis	Dependencia de runtime	Única dependencia externa; no hay base de datos, Kafka ni llamadas HTTP salientes.

Tabla 5. Integraciones del gateway.

El acoplamiento con `promotion-service` es **implícito**: ambos comparten el prefijo `user:status:` y el significado de los valores, pero no existe un contrato tipado ni un catálogo común de estados. Esa ausencia explica el hallazgo crítico H-01.

8. Pruebas existentes

Clase	Casos	Cobertura funcional
GateControllerTest	1	Verifica que POST /validate devuelve 200 y el JSON esperado con el servicio simulado.
QrValidationServiceTest	2	Token válido con estado CLEAR → GREEN; token con estado CONTAGIED → RED.

Tabla 6. Pruebas automatizadas actuales (3 casos).

Casos no cubiertos relevantes:

- Token expirado, malformado, con firma inválida o ausente en el cuerpo.
- Matriz completa de estados: ACTIVE, SUSPECT, PROBABLE, CONFIRMED, RECOVERED y null.
- Caída, timeout o respuesta vacía de Redis.
- Integración real con Redis (Testcontainers) y con un JWT emitido por auth-service.
- Verificación de autenticación y del permiso gate:scan.
- Cobertura de líneas: el proyecto exige ≥ 60 % para gateway en el entregable E1.6; no hay JaCoCo configurado.

8.1 Ejecución y limitaciones

```
./gradlew :services:circleguard-gateway-service:test
```

Las pruebas actuales usan Mockito y MockMvc, por lo que no requieren Redis ni infraestructura. Al no existir pruebas de integración, la suite no valida la conexión real con Redis ni el contrato con auth-service; una regresión en el formato del token o en el nombre de la clave pasaría inadvertida.

9. Hallazgos

Los hallazgos se clasifican por severidad según su impacto actual sobre la contención sanitaria, la seguridad y la operación. Las referencias apuntan al código que los evidencia.

ID	Severidad	Hallazgo y evidencia	Impacto
H-01	Crítico	**Vocabulario de estados incompatible entre gateway y promotion** QrValidationService.java:37 frente a HealthStatusService.java:98-105. El gateway solo deniega CONTAGIED y POTENTIAL; promotion escribe SUSPECT, PROBABLE y CONFIRMED.	Un caso confirmado o un contacto de riesgo recibe GREEN y puede ingresar al campus.
H-02	Crítico	**Un estado ausente en Redis permite el acceso** QrValidationService.java:35-41. Si la clave no existe, status es null y el flujo llega a la respuesta GREEN.	Falla abierta ante usuarios sin estado materializado, claves evictadas o datos incompletos.
H-03	Crítico	**Endpoint sin autenticación ni autorización** build.gradle.kts no incluye Spring Security. El Bearer del cliente se ignora y el permiso gate:scan no se valida.	Cualquier cliente que alcance el puerto puede validar tokens; no hay control de quién escanea.
H-04	Crítico	**El QR configurado expira en 300 milisegundos** auth-service QrTokenService.java:27 suma expiration como milisegundos; application.yml define qr.expiration: 300. QrTokenController responde expiresIn: 60.	Casi todos los QR llegan expirados al gateway y el flujo de ingreso falla de forma intermitente.
H-05	Alto	**La caída de Redis se confunde con un token inválido** QrValidationService.java:43-45 captura cualquier excepción y devuelve el mensaje 'Invalid or Expired Token'.	No se distingue indisponibilidad de seguridad; se enmascara la causa y se impide alertar correctamente.
H-06	Alto	**Sin resiliencia ni timeouts hacia Redis** No hay Resilience4j, configuración de timeouts, retry, circuit breaker ni bulkhead.	Una latencia o caída de Redis bloquea las validaciones y afecta la operación de portería.
H-07	Alto	**Secretos en texto plano y versionados** qr.secret y jwt.secret aparecen en application.yml de gateway y auth-service; también en bin/main.	Incumple la gestión segura de secretos esperada en E2.6 y permite forjar tokens si el repositorio se filtra.
H-08	Medio	**Ausencia de observabilidad y auditoría** No hay Actuator, Micrometer, métricas, logs estructurados ni registro de escaneos (gate, staff, resultado, timestamp).	No se puede medir throughput, latencia, tasa de denegación ni reconstruir un ingreso.
H-09	Medio	**Sin empaquetamiento ni pipeline** No existe Dockerfile, healthcheck, entrada en docker-compose.dev.yml, OpenAPI ni configuración de CI/JaCoCo.	El servicio no cumple aún los entregables E1.3, E1.4 y E1.6 para gateway.
H-10	Medio	**Cobertura de pruebas insuficiente** Solo 3 casos; no se prueban expiración, estados de riesgo, null, Redis caído ni autenticación.	Los defectos críticos H-01, H-02 y H-04 no serían detectados por la suite actual.
H-11	Bajo	**Artefacto de compilación versionado** bin/main/application.yml está en Git y duplica la configuración de src/main/resources.	Ruido, riesgo de configuración divergente y confusión sobre la fuente de verdad.
H-12	Bajo	**Desalineación en el cliente Expo** useGateValidation.ts importa API_BASE_URL, pero constants/Config.ts solo exporta GATEWAY_BASE_URL.	Posible error de compilación o URL indefinida en el flujo de escaneo.
H-13	Bajo	**Plugins Kotlin en un módulo solo Java** build.gradle.kts aplica kotlin("jvm") y kotlin("plugin.spring"); no hay archivos .kt.	Dependencias y tiempos de build innecesarios; configuración menos clara.

Tabla 7. Inventario de hallazgos priorizado por severidad.

9.1 Matriz de decisión actual frente a la esperada

Estado en Redis	Gateway actual	Esperado	Resultado
ACTIVE	GREEN	GREEN	Correcto
SUSPECT	GREEN	RED	Falla de contención
PROBABLE	GREEN	RED	Falla de contención
CONFIRMED	GREEN	RED	Falla crítica
RECOVERED	GREEN	GREEN*	Correcto según política
null / desconocido	GREEN	RED	Falla abierta
CONTAGIED	RED	No se usa	Valor legado
POTENTIAL	RED	No se usa	Valor legado

Tabla 8. Matriz de decisión. * RECOVERED puede permitirse si la política sanitaria lo determina.

10. Recomendaciones

Las acciones se ordenan por prioridad. Las de prioridad P0 corrigen el comportamiento inseguro actual y deberían aplicarse antes de cualquier demostración o despliegue.

Prioridad	Acción	Detalle
P0	**Unificar el catálogo de estados y corregir la matriz**	Definir un contrato o enum compartido (ACTIVE, SUSPECT, PROBABLE, CONFIRMED, RECOVERED). Denegar SUSPECT, PROBABLE y CONFIRMED; denegar null y valores desconocidos; documentar la política para RECOVERED.
P0	**Corregir la vigencia del QR**	Usar una unidad consistente: expiration en segundos y multiplicar por 1000 al construir la fecha, o migrar a Duration. Alinear expiresIn del controlador con el TTL real.
P0	**Añadir pruebas de la matriz y de tokens**	Cubrir los seis estados, null, token expirado, firma inválida, cuerpo vacío y Redis caído. Configurar JaCoCo y exigir ≥ 60 % de líneas.
P1	**Autenticar y autorizar el endpoint**	Incorporar Spring Security como resource server, validar el JWT del empleado y exigir el permiso gate:scan. Añadir rate limiting por IP y por empleado.
P1	**Fortalecer el token QR**	Incluir jti de un solo uso, issuer y audience; validar que el token sea de tipo QR; rotar el secreto y eliminarlo del repositorio.
P1	**Externalizar la configuración y los secretos**	Mover qr.secret, jwt.secret y las URLs a variables de entorno, perfiles de Spring y Secrets de Kubernetes. Eliminar las propiedades no utilizadas.
P2	**Introducir resiliencia**	Envolver la consulta a Redis con Resilience4j (time limiter, retry y circuit breaker). Diferenciar 401/403 por token o estado y 503 por dependencia no disponible.
P2	**Instrumentar el servicio**	Añadir Actuator y Micrometer; exponer métricas de validaciones, latencia, denegaciones y errores. Registrar auditoría estructurada (gate, staff, anonymousId, resultado, timestamp).
P2	**Publicar el contrato**	Añadir springdoc-openapi y documentar el endpoint, los códigos de respuesta y el esquema de ValidationResult.
P3	**Completar la cadena DevOps**	Dockerfile multi-stage con usuario no root, healthcheck, servicio en docker-compose por ambiente, stage de CI con pruebas y publicación de imagen, y sondas liveness/readiness para Kubernetes.
P3	**Limpiar el módulo**	Eliminar bin/ del control de versiones, retirar los plugins Kotlin si no se usarán y alinear el cliente Expo (GATEWAY_BASE_URL).

Tabla 9. Plan de mejora priorizado.

11. Conclusión

El microservicio Gateway está bien delimitado y es fácil de entender: una ruta, una responsabilidad y una dependencia externa. Esa simplicidad es una fortaleza para el pipeline y para la operación. Sin embargo, la lógica de decisión actual **no protege el campus**: permite el ingreso de estados de riesgo y falla abierto cuando falta el estado en Redis. A esto se suman un TTL de QR mal configurado, un endpoint sin autenticación y la ausencia de resiliencia, observabilidad y empaquetamiento.

La corrección es acotada: un catálogo común de estados, una matriz de decisión explícita, pruebas que la cubran y tres ajustes de configuración. Con esas mejoras, el servicio puede cumplir su papel dentro de CircleGuard y alinearse con los entregables de la Parte 1 del proyecto (empaquetamiento, CI y cobertura) y de la Parte 2 (resiliencia, configuración externa, observabilidad y seguridad).

> **Recomendación inmediata**
>
> No desplegar el gateway en un ambiente compartido hasta corregir H-01, H-02, H-03 y H-04. Son cambios pequeños pero de impacto directo sobre la seguridad sanitaria y la validez del flujo de ingreso.

Anexo. Archivos revisados

services/circleguard-gateway-service/
  `build.gradle.kts`

services/circleguard-gateway-service/src/main/resources/
  `application.yml`

services/circleguard-gateway-service/src/main/java/com/circleguard/gateway/
  `GatewayServiceApplication.java`

services/circleguard-gateway-service/src/main/java/com/circleguard/gateway/controller/
  `GateController.java`

services/circleguard-gateway-service/src/main/java/com/circleguard/gateway/service/
  `QrValidationService.java`

services/circleguard-gateway-service/src/test/java/com/circleguard/gateway/controller/
  `GateControllerTest.java`

services/circleguard-gateway-service/src/test/java/com/circleguard/gateway/service/
  `QrValidationServiceTest.java`

services/circleguard-auth-service/src/main/java/com/circleguard/auth/service/
  `QrTokenService.java`

services/circleguard-auth-service/src/main/java/com/circleguard/auth/controller/
  `QrTokenController.java`

services/circleguard-auth-service/src/main/resources/
  `application.yml`

services/circleguard-promotion-service/src/main/java/com/circleguard/promotion/service/
  `HealthStatusService.java`

services/circleguard-promotion-service/src/main/java/com/circleguard/promotion/service/
  `StatusLifecycleService.java`

mobile/hooks/
  `useGateValidation.ts`

mobile/constants/
  `Config.ts`

`docker-compose.dev.yml`

`settings.gradle.kts`

`build.gradle.kts`